

Persistance & Intégrité des données — CanBankX

Auteur : Pascal Bourgoïn **Date :** Mars 2026 **Compléments :** [arc42.md](#) §8.3–8.5 · [ADR002–microservices](#) · [ADR003–ledger](#) · [ADR006–cache](#)

Table des matières

1. Schémas MySQL isolés
 2. Modèle de données par service
 3. Configuration JPA / HikariCP
 4. Opérations atomiques sur les soldes
 5. Redis — double usage TTL
 6. Mécanisme exactly-once (idempotence des paiements)
 7. Journal d'audit append-only
 8. Compensation sur panne partielle
 9. Propagation des transactions Spring
 10. Risques et dette technique
-

1. Schémas MySQL isolés

Un seul conteneur MySQL 8.4 héberge **trois schémas logiquement isolés**, un par microservice. L'isolation est appliquée au niveau des grants MySQL — l'utilisateur applicatif [projet](#) n'a accès qu'à son propre schéma.

1.1 Initialisation ([db-init/init.sql](#))

```
CREATE DATABASE IF NOT EXISTS db_identity CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
CREATE DATABASE IF NOT EXISTS db_account CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
CREATE DATABASE IF NOT EXISTS db_payment CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'projet'@'%' IDENTIFIED BY 'projet';
ALTER USER 'projet'@'%' IDENTIFIED BY 'projet';

GRANT ALL PRIVILEGES ON db_identity.* TO 'projet'@'%';
GRANT ALL PRIVILEGES ON db_account.* TO 'projet'@'%';
GRANT ALL PRIVILEGES ON db_payment.* TO 'projet'@'%';

FLUSH PRIVILEGES;
```

Ce script est exécuté par le service Docker [db-init](#) (one-shot) au démarrage de l'infrastructure. Il est idempotent ([IF NOT EXISTS](#)) et peut être rejoué sans risque.

1.2 Mapping service → schéma

Service	Schéma	Tables
identity-service	db_identity	clients
account-service	db_account	accounts
payment-service	db_payment	bank_transactions, audit_log

1.3 Principe d'isolation

Il n'existe aucune clé étrangère (FK) cross-schéma. Les services se réfèrent par des identifiants métier :

- `account-service` stocke `client_id` (UUID) — référence logique vers `db_identity.clients`
- `payment-service` stocke `source_account_number` et `target_account_number` (String) — référence logique vers `db_account.accounts`

La cohérence inter-services est maintenue par les appels HTTP REST et non par des contraintes SQL distribuées. Cette décision accepte la consistance éventuelle au profit de l'indépendance des schémas.

2. Modèle de données par service

2.1 `db_identity.clients`

Colonne	Type	Contrainte
<code>id</code>	<code>VARCHAR(36)</code>	PK
<code>first_name</code>	<code>VARCHAR(100)</code>	NOT NULL
<code>last_name</code>	<code>VARCHAR(100)</code>	NOT NULL
<code>email</code>	<code>VARCHAR(255)</code>	UNIQUE, NOT NULL
<code>password_hash</code>	<code>VARCHAR(255)</code>	NOT NULL — BCrypt 8
<code>nas</code>	<code>VARCHAR(9)</code>	UNIQUE, NOT NULL
<code>phone_number</code>	<code>VARCHAR(20)</code>	nullable
<code>address</code>	<code>TEXT</code>	nullable
<code>status</code>	<code>ENUM('PENDING', 'ACTIVE', 'SUSPENDED')</code>	NOT NULL
<code>created_at</code>	<code>DATETIME</code>	NOT NULL

Les contraintes UNIQUE sur `email` et `nas` sont gérées directement par JPA (`@Column(unique=true)`) et produisent une `DuplicateEmailException` / `DuplicateNasException` interceptée par `GlobalExceptionHandler` → HTTP 409.

2.2 `db_account.accounts`

Colonne	Type	Contrainte
id	VARCHAR(36)	PK
account_number	VARCHAR(20)	UNIQUE, NOT NULL
client_id	VARCHAR(36)	NOT NULL — référence logique
type	ENUM('CHECKING', 'SAVINGS')	NOT NULL
balance	DECIMAL(15,2)	NOT NULL, DEFAULT 0
created_at	DATETIME	NOT NULL

Le `account_number` est généré aléatoirement par `AccountService` avec vérification d'unicité :

```
private String generateUniqueAccountNumber() {
    String candidate;
    do {
        candidate = String.valueOf(1_000_000_000L +
                                   (long)(RANDOM.nextDouble() * 9_000_000_000L));
    } while (accountRepository.existsByAccountNumber(candidate));
    return candidate;
}
```

2.3 `db_payment.bank_transactions`

Colonne	Type	Contrainte
id	VARCHAR(36)	PK
idempotency_key	VARCHAR(255)	UNIQUE , NOT NULL
source_account_number	VARCHAR(20)	NOT NULL, INDEX
target_account_number	VARCHAR(20)	nullable (DEBIT/CREDIT), INDEX
amount	DECIMAL(19,4)	NOT NULL
type	ENUM('DEBIT', 'CREDIT', 'TRANSFER')	NOT NULL
status	ENUM('PENDING', 'COMPLETED', 'FAILED')	NOT NULL
audit_note	VARCHAR(512)	nullable
created_at	DATETIME	NOT NULL
last_modified_at	DATETIME	auto-update

Index déclarés via `@Table(indexes = {...})` :

```
@Index(name = "idx_idempotency_key", columnList = "idempotency_key",
unique = true),
@Index(name = "idx_source_account", columnList =
"source_account_number"),
@Index(name = "idx_target_account", columnList =
"target_account_number"),
@Index(name = "idx_status_created", columnList = "status, created_at")
```

2.4 db_payment.audit_log

Colonne	Type	Contrainte
id	VARCHAR(36)	PK
transaction_id	VARCHAR(36)	NOT NULL, INDEX — référence logique vers <code>bank_transactions</code>
action	VARCHAR(64)	NOT NULL
detail	VARCHAR(1024)	nullable
created_at	DATETIME	NOT NULL, INDEX

Toutes les colonnes sont déclarées `updatable = false` sur l'entité JPA — aucun `UPDATE` n'est jamais émis sur cette table.

3. Configuration JPA / HikariCP

Chaque service déclare sa datasource dans `application.yaml` :

```
spring:
  datasource:
    url: jdbc:mysql://mysql:3306/db_payment?
useSSL=false&serverTimezone=UTC
    username: ${DB_USER:projet}
    password: ${DB_PASSWORD:projet}
    hikari:
      maximum-pool-size: 50
      minimum-idle: 5
      connection-timeout: 30000
      idle-timeout: 600000
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    properties:
      hibernate.dialect: org.hibernate.dialect.MySQLDialect
```

Pool HikariCP à 50 connexions : ce paramètre est le facteur limitant observé sous stress test. Chaque instance peut maintenir 50 connexions MySQL simultanées. En mode LB (N=2), le total monte à 100 connexions disponibles pour `payment-service`.

ddl-auto: update : Hibernate génère automatiquement les tables et index au démarrage si absents. Acceptable pour le contexte académique ; en production il faudrait migrer vers Flyway pour des migrations versionnées et reproductibles.

4. Opérations atomiques sur les soldes

La règle d'intégrité fondamentale est : **un solde ne peut jamais devenir négatif**. Cette garantie est implémentée avec une requête JPQL atomique dans `AccountRepository` :

```
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("UPDATE Account a SET a.balance = a.balance - :amount " +
        "WHERE a.accountNumber = :accountNumber AND a.balance >= :amount")
int atomicDebit(@Param("accountNumber") String accountNumber,
                @Param("amount") BigDecimal amount);
```

Cette requête se traduit en SQL :

```
UPDATE accounts
SET balance = balance - ?
WHERE account_number = ?
AND balance >= ?
```

Propriétés de cette approche :

Propriété	Explication
Atomique	L'opération est un seul <code>UPDATE</code> — pas de <code>SELECT + UPDATE</code> qui ouvrirait une fenêtre de race condition
Optimiste	Pas de <code>SELECT FOR UPDATE</code> / verrou pessimiste — InnoDB utilise un row-level lock uniquement pendant le <code>UPDATE</code>
Auto-vérification	Si <code>balance < amount</code> , aucune ligne n'est modifiée → <code>rowsUpdated == 0</code> → <code>InsufficientFundsException</code>
Résistant aux concurrents	Deux appels simultanés sur le même compte : InnoDB sérialise les row-locks, le second voit le solde mis à jour par le premier

La valeur de retour (`int`) est vérifiée dans `AccountService` :

```
int updated = accountRepository.atomicDebit(accountNumber, amount);
if (updated == 0) {
    if (!accountRepository.findByAccountNumber(accountNumber).isPresent())
```

```
{
    throw new AccountNotFoundException(accountNumber);    // 404
}
throw new InsufficientFundsException(accountNumber);    // 422
}
```

Le crédit (`atomicCredit`) est similaire mais sans la condition `AND balance >= :amount` — un crédit ne peut pas échouer par manque de fonds.

5. Redis — double usage TTL

Un seul conteneur Redis 7 sert deux usages avec des namespaces de clés et des TTL distincts.

5.1 Tokens MFA (`identity-service`)

Propriété	Valeur
Clé	<code>mfa:challenge:{challengeToken}</code> — le token est un UUID aléatoire
Valeur	<code>clientId</code> (UUID String)
TTL	5 minutes
Opérations	<code>SET ... EX 300</code> au login, <code>GET</code> à la vérification MFA, <code>DEL</code> immédiat après validation (usage unique)

Le token est **détruit dès la première validation réussie** — un attaquant qui intercepte le token ne peut pas l'utiliser une deuxième fois.

5.2 Clés d'idempotency (`payment-service`)

Propriété	Valeur
Clé	<code>payment:idem:{idempotencyKey}</code> — la clé est fournie par le client HTTP
Valeur	UUID de la <code>BankTransaction</code> correspondante
TTL	24 heures
Opérations	<code>GET</code> en début de traitement, <code>SET ... EX 86400</code> après COMPLETED

5.3 Configuration Redis

```
# dans docker-compose.yml
command: ["redis-server",
  "--appendonly", "yes",
  "--maxmemory", "256mb",
  "--maxmemory-policy", "volatile-lru"
]
```

appendonly yes assure la persistance sur disque des clés Redis entre les redémarrages. **volatile-lru** évite l'éviction des clés sans TTL (inexistantes ici) et privilégie l'éviction des clés les plus anciennes en cas de pression mémoire.

6. Mécanisme exactly-once (idempotence des paiements)

L'objectif est de garantir qu'une transaction est exécutée **exactement une fois**, même si le client HTTP renvoie la même requête plusieurs fois (réseau instable, timeout client, retry automatique).

6.1 Double filet

```
Filet 1 : Redis    - vérification rapide (~1 ms)
Filet 2 : MySQL    - contrainte UNIQUE sur idempotency_key
```

Les deux filets sont indépendants. Si Redis redémarre et perd ses clés, MySQL prend le relais.

6.2 Algorithme complet (**PaymentService.submit**)

```
1. REDIS GET payment:idem:{key}
   | HIT → retourner la transaction existante (aucune écriture DB)
   | MISS → continuer

2. DB SELECT findByIdempotencyKey(key) ← fallback si Redis a
   redémarré
   | COMPLETED → recacher dans Redis, retourner la transaction
   | PENDING → lever IllegalStateException (concurrency)
   | FAILED → supprimer et autoriser un retry
   | null → continuer (vraiment nouvelle transaction)

3. DB INSERT bank_transaction (status=PENDING) ←
   Propagation.REQUIRES_NEW
   + AuditLog INSERT TRANSFER_INITIATED

4. Exécution métier
   | AccountClient.debit(source, amount)
   | → UPDATE accounts SET balance=balance-amount WHERE
   | balance>=amount
   | [si TRANSFER] AccountClient.credit(target, amount)
   | → UPDATE accounts SET balance=balance+amount
   | [si CREDIT] AccountClient.credit(source, amount)

5. DB UPDATE bank_transaction → COMPLETED
   + AuditLog INSERT TRANSFER_COMPLETED

6. REDIS SET payment:idem:{key} = txId EX 86400

7. CompletableFuture.runAsync → sendConfirmationEmail() [fire-and-forget]
```

```
← 201 Created {id, status: COMPLETED, ...}
```

6.3 Résistance à la concurrence

La contrainte `UNIQUE(idempotency_key)` en base garantit que même si deux requêtes identiques arrivent simultanément et passent toutes deux le check Redis en même temps (race condition), l'une des deux échouera sur l'`INSERT` avec une violation de contrainte. Cette exception est remontée au client comme erreur 409 ou 500.

7. Journal d'audit append-only

L'entité `AuditLog` est le registre immuable de chaque étape d'une transaction. Elle implémente un pattern **Ledger** : on n'écrit que vers l'avant, jamais en arrière.

7.1 Garanties JPA

L'entité est déclarée avec `updatable = false` sur **toutes** ses colonnes :

```
@Column(updatable = false, nullable = false)
private UUID transactionId;

@Column(updatable = false, nullable = false, length = 64)
private String action;

@Column(updatable = false, length = 1024)
private String detail;

@CreatedDate
@Column(updatable = false, nullable = false)
private Instant createdAt;
```

Hibernate ne peut physiquement pas émettre un `UPDATE` sur ces colonnes — toute tentative de modification est silencieusement ignorée. `AuditLogRepository` ne contient aucune méthode `deleteBy*` ou `update*`.

7.2 Actions tracées

Action	Déclencheur
<code>TRANSFER_INITIATED</code>	<code>persistPending()</code> — transaction créée en PENDING
<code>BALANCE_DEBITED</code>	Débit du compte source réussi
<code>BALANCE_CREDITED</code>	Crédit du compte destination réussi
<code>TRANSFER_COMPLETED</code>	<code>completeTransaction()</code> — statut COMPLETED
<code>TRANSFER_FAILED</code>	<code>failTransaction()</code> — exception catchée

Action	Déclencheur
COMPENSATION_CREDIT	Crédit de compensation appliqué après échec du TRANSFER

7.3 Propagation **REQUIRES_NEW**

Chaque étape d'audit est persistée dans sa **propre transaction Spring** :

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
protected void auditStep(UUID txId, String action, String detail) {
    auditLogRepository.save(AuditLog.builder()
        .transactionId(txId).action(action).detail(detail).build());
}
```

REQUIRES_NEW suspend la transaction parente et ouvre une transaction indépendante. Cela garantit que l'entrée d'audit est committée en base **même si la transaction principale est rollbackée**. Un journal d'audit qui disparaît avec le rollback n'aurait aucune valeur de traçabilité.

8. Compensation sur panne partielle

Un TRANSFER comporte deux opérations sur deux comptes différents. Si le **débit réussit** mais que le **crédit échoue** (compte destination introuvable, service down), le solde du compte source doit être restauré.

```
try {
    accountClient.debit(sourceAccount, amount, txRef);
    sourceDebited = true; // débit confirmé
    // ...
    accountClient.credit(targetAccount, amount, txRef);
    // ...
} catch (Exception ex) {
    if (sourceDebited && dto.getType() == TransactionType.TRANSFER) {
        // Compensation : re-créditer le compte source
        accountClient.credit(sourceAccount, amount, txRef + "-
COMPENSATION");
        auditStep(txId, "COMPENSATION_CREDIT",
            "Compensation credit applied after failed transfer.");
    }
    failTransaction(txId, ex.getMessage());
    throw ex; // re-propagation pour retourner le bon code HTTP au
client
}
```

Propriétés de la compensation :

Propriété	Détail
-----------	--------

Propriété	Détail
Automatique	Déclenchée sans intervention humaine dans le même thread de traitement
Tracée	L'action <code>COMPENSATION_CREDIT</code> est enregistrée dans <code>AuditLog</code>
Transaction finale	La <code>BankTransaction</code> passe à <code>FAILED</code> , pas à <code>COMPLETED</code>
Idempotente	La clé <code>txRef</code> + <code>"-COMPENSATION"</code> garantit que la compensation ne s'applique qu'une fois
Limitation	Si la compensation elle-même échoue, un log <code>CRITICAL</code> est émis — aucun retry automatique

En cas d'échec de la compensation, la cohérence doit être restaurée manuellement. Ce cas extrême est documenté comme risque dans `arc42.md §11`.

9. Propagation des transactions Spring

`PaymentService` utilise plusieurs niveaux de propagation pour isoler les étapes critiques :

Méthode	Propagation	Raison
<code>submit()</code>	Par défaut (<code>REQUIRED</code>)	Transaction principale englobant tout le flow
<code>persistPending()</code>	<code>REQUIRES_NEW</code>	INSERT PENDING engagé immédiatement — visible même si la transaction principale rollbacke
<code>auditStep()</code>	<code>REQUIRES_NEW</code>	Chaque audit line engagée indépendamment — ne disparaît pas si le parent rollbacke
<code>completeTransaction()</code>	<code>REQUIRES_NEW</code>	UPDATE COMPLETED engagé séparément — atomique avec l'audit final
<code>failTransaction()</code>	<code>REQUIRES_NEW</code>	UPDATE FAILED engagé même si une exception est en cours de propagation
<code>getId()</code> / lectures	<code>readOnly = true</code>	Optimisation Hibernate — pas de dirty check, pas de snapshot

`AccountService` utilise `@Transactional` sur toutes les méthodes d'écriture.

`@Transactional(readOnly = true)` est systématiquement appliqué aux méthodes de lecture pour que Hibernate désactive le dirty checking et que Spring puisse router les lectures vers un read replica si disponible.

10. Risques et dette technique

Risque	Sévérité	État actuel	Mitigation en production
--------	----------	-------------	--------------------------

Risque	Sévérité	État actuel	Mitigation en production
SPOF MySQL	Haute	Volumes Docker persistants seulement	MySQL Group Replication, RDS Multi-AZ
ddl-auto: update	Moyenne	Fonctionnel en développement	Migrer vers Flyway pour des migrations versionnées
Compensation non-idempotente	Haute	Clé —COMPENSATION unique par tx	Saga pattern avec état persisté en base
SPOF Redis	Faible	Fallback DB maintient la correction	Redis Sentinel ou Redis Cluster
Pas de retry sur la compensation	Haute	Log CRITICAL + intervention manuelle	Dead letter queue + job de réconciliation
Contention InnoDB à 200 VUs	Haute	Résolu par pool de 20 comptes	Partitionnement des données, sharding
Pool HikariCP saturé à 150+ VUs	Haute	Observable dans Grafana	Mode LB (100 connexions totales), puis read replica